Implementation of Spiteri et al. (2005) trapping and Hysteresis Model as well as alternative implementation of the Carlson hysteresis model

This document summarizes all steps, file modifications, and new files required to implement the Spiteri trapping and hysteresis model, including the addition of a new keyword,'SPHYST,' into OPM Flow. Additionally, an alternative implementation of the Carlson hysteresis model is added, in which the flowing and trapped saturations are computed explicitly, and the gas relative permeability is evaluated directly from the primary drainage (gas injection).

==========

1. OPM FLOW FILES AND SETUP

==========

1.1 Open PowerShell and run:

```
wsl -install
```

1.2 Restart your PC and open Ubuntu from the Start menu.

1.3 Update Ubuntu packages:

```
sudo apt update && sudo apt upgrade -y
```

1.4 Run the following commands in Ubuntu:

```
sudo apt-get install -y software-properties-common build-essential gfortran
cmake git pkg-config

sudo add-apt-repository -y ppa:opm/ppa

sudo apt-get update -y

sudo apt-get install -y mpi-default-dev openmpi-bin \

    libblas-dev liblapack-dev libsuitesparse-dev zlib1g-dev \

    libeigen3-dev libcgal-dev libboost-all-dev libfmt-dev libcjson-dev \

    libdune-common-dev libdune-geometry-dev libdune-istl-dev libdune-grid-dev
```

1.5 Download OPM Source Code

```
rm -rf ~/opm && mkdir -p ~/opm && cd ~/opm
```

```
git clone https://github.com/OPM/opm-common.git

git clone https://github.com/OPM/opm-grid.git

git clone https://github.com/OPM/opm-simulators.git
```

==========

2. NEW FILES CREATED

==========

2.1 File: SPHYST.json
Path:
~/opm/opm-common/opm/input/eclipse/share/keywords/000_Eclipse100/S/SPHYST.json
~/opm/opm-common/opm/input/eclipse/share/keywords/900_OPM/S/SPHYST.json

Purpose:
Defines the SPHYST keyword in JSON form (the format used by OPM).

Content:

```
{
  "name": "SPHYST",
  "sections": ["PROPS"],
  "size": 1,
  "items": [
    {
      "item": 1,
      "name": "AlphaSP",
      "value_type": "DOUBLE",
      "default": 0.0,
      "min": 0.0,
      "max": 1.0,
      "dimension": "1",
      "comment": "0 ≤ AlphaSP ≤ 1"
    },
    {
      "item": 2,
      "name": "BetaSP",
      "value_type": "DOUBLE",
      "default": 0.0,
      "min": 0.0,
      "dimension": "1",
      "comment": "BetaSP ≥ 0"
    },
    {
      "item": 3,
      "name": "GammaSP",
      "value_type": "DOUBLE",
      "default": 0.0,
      "min": -2.0,
      "max": 3.0,
      "dimension": "1",
      "comment": "-2 ≤ GammaSP ≤ 3"
    },
```

```
    {
      "item": 4,
      "name": "ShiftSP",
      "value_type": "INT",
      "default": 0,
      "min": 0,
      "max": 1,
      "comment": "ShiftSP ∈ {0, 1}"
    }
  ]
}
```

2.2 File: SphystConfig.hpp
Path:
~/opm/opm-simulators/src/sphyst/SphystConfig.hpp

Purpose:
Reads the SPHYST keyword from the deck using OPM's Deck API.

Content:

```
#pragma once

#include <opm/input/eclipse/Deck/Deck.hpp>
#include <opm/input/eclipse/Deck/DeckView.hpp>
#include <opm/input/eclipse/Deck/DeckKeyword.hpp>
#include <opm/input/eclipse/Deck/DeckRecord.hpp>
#include <opm/input/eclipse/Deck/DeckItem.hpp>

struct SphystConfig {
    double alphaSP = 0.0;
    double betaSP  = 0.0;
    double gammaSP = 0.0;
    double shiftSP = 0.0;
    bool   present = false;
};

inline SphystConfig readSphyst(const Opm::Deck& deck)
{
    SphystConfig cfg;

    // If the keyword is not present, just return defaults.
    if (!deck.hasKeyword("SPHYST"))
        return cfg;

    // Deck -> DeckView of the keyword
    const auto& kwView = deck["SPHYST"];   // DeckView<DeckKeyword>

    // First occurrence of SPHYST
    const auto& kw = kwView[0];            // DeckKeyword

    // First record of that keyword
    const auto& rec = kw.getRecord(0);     // DeckRecord

    // Items 0..3 correspond to AlphaSP, BetaSP, GammaSP, ShiftSP.
```

```
    // In this OPM version, DeckItem exposes getSIDouble(0) for DOUBLE values.
    cfg.alphaSP = rec.getItem(0).getSIDouble(0);  // 0 ≤ AlphaSP ≤ 1
    cfg.betaSP  = rec.getItem(1).getSIDouble(0);  // BetaSP ≥ 0
    cfg.gammaSP = rec.getItem(2).getSIDouble(0);  // -2 ≤ GammaSP ≤ 3

    const int shiftInt = rec.getItem("ShiftSP").get<int>(0);
    cfg.shiftSP = static_cast<double>(shiftInt);


    cfg.present = true;
    return cfg;
}
```

2.3 File: EclSphystGlobal.hpp
Path:
~/opm/opm-common/opm/material/fluidmatrixinteractions/EclSphystGlobal.hpp

Purpose:
Stores SPHYST values globally so both Params and Law can read them.

Content:

```
#pragma once

namespace Opm {

struct EclSphystGlobal {
    inline static bool   enabled = false;  // C++17 inline static
    inline static double alphaSP = 0.0;
    inline static double betaSP  = 0.0;
    inline static double gammaSP = 0.0;
    inline static double shiftSP = 0.0;
};

} // namespace Opm
```

==========
3. MODIFIED EXISTING FILES
==========

3.1 File: keyword_list
Path:
~/opm/opm-simulators/src/sphyst/keyword_list.hpp

Modification:
Adds the SPHYST to keyword list.

3.2 File: readDeck.cpp
Path:
~/opm/opm-simulators/opm/simulators/utils/readDeck.cpp

Modification:

Change 1: Lines 31-33 (New Includes)

Location: After mpi.h endif
Change: Added SPHYST configuration includes

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`#endif`
 	
`#include <opm/simulators/utils/readDeck.hpp>`
	`#endif`
	
	`#include "src/sphyst/SphystConfig.hpp"`
	`#include <opm/material/fluidmatrixinteractions/EclSphystGlobal.hpp>`
	
	`#include <opm/simulators/utils/readDeck.hpp>`

Change 2: Lines 369-407 (SPHYST Integration)

Location: After deck reading, before eclipseState check
Change: Complete SPHYST keyword reading and validation

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`const auto deck = readDeckFile(deckFilename, checkDeck, parser,`
 	
`*parseContext, treatCriticalAsNonCritical, errorGuard);`
 	
`if (eclipseState == nullptr) {`
	`const auto deck = readDeckFile(deckFilename, checkDeck, parser,`
	
	`*parseContext, treatCriticalAsNonCritical, errorGuard);`
	
	`// ----------`
	`// SPHYST integration: read SPHYST keyword and set globals`
	`// ----------`
	`{`
	`  auto sph = readSphyst(deck);`
	
	`  // Clamp ranges to the keyword constraints`
	`  if (sph.present) {`
	`  if (sph.alphaSP < 0.0) sph.alphaSP = 0.0;`
	`  if (sph.alphaSP > 1.0) sph.alphaSP = 1.0;`
	
	`  if (sph.betaSP < 0.0) sph.betaSP = 0.0;`
	
	`  if (sph.gammaSP < -2.0) sph.gammaSP = -2.0;`
	`  if (sph.gammaSP >  3.0) sph.gammaSP =  3.0;`
	
	`  sph.shiftSP = (sph.shiftSP >= 0.5) ? 1.0 : 0.0;`
	`  }`
	
	`  // Write to global SPHYST storage`
	`  Opm::EclSphystGlobal::enabled = sph.present;`
	`  Opm::EclSphystGlobal::alphaSP = sph.alphaSP;`
	`  Opm::EclSphystGlobal::betaSP  = sph.betaSP;`
	`  Opm::EclSphystGlobal::gammaSP = sph.gammaSP;`
	`  Opm::EclSphystGlobal::shiftSP = sph.shiftSP;`
	
	`  if (sph.present) {`
	`  std::cout << "SPHYST read: "`

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

	`<< "AlphaSP=" << sph.alphaSP`
	`<< " BetaSP=" << sph.betaSP`
	`<< " GammaSP=" << sph.gammaSP`
	`<< " ShiftSP=" << sph.shiftSP`
	`<< std::endl;`
	`  }`
	`}`
	`// ----------`
	`if (eclipseState == nullptr) {`

Change 3 : Lines 753-755 (SPHYST Integration)

Location: After deck reading, before eclipseState check
Change: Broadcast the SPHYST globals from rank 0 to every other rank.

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`#if HAVE_MPI`
`    else {`
 	
`defineStateObjectsOnNonIORank(comm, std::move(python), eclipseState,`
 	
`schedule, udqState, actionState, wtestState,`
 	
`summaryConfig);`
`    }`
 	
`    // In case of parse errors eclipseState/schedule might be null and`
`    // trigger segmentation faults in parallel during broadcast (e.g. when`
`    // serializing the non-existent TableManager)`
	`#if HAVE_MPI`
	`    else {`
	
	`defineStateObjectsOnNonIORank(comm, std::move(python), eclipseState,`
	
	`schedule, udqState, actionState, wtestState,`
	
	`summaryConfig);`
	`    }`
	
	`    // ---------------------------------------------------------------------------`
	`    // Broadcast SPHYST globals from I/O rank (0) to all other ranks`
	`    // ---------------------------------------------------------------------------`
	`    {`
	`        // Pack enabled flag and 4 doubles into an array`
	`        double sphyst_buf[5];`
	
	`        if (comm.rank() == 0) {`
	`            sphyst_buf[0] = Opm::EclSphystGlobal::enabled ? 1.0 : 0.0;`
	`            sphyst_buf[1] = Opm::EclSphystGlobal::alphaSP;`
	`            sphyst_buf[2] = Opm::EclSphystGlobal::betaSP;`
	`            sphyst_buf[3] = Opm::EclSphystGlobal::gammaSP;`
	`            sphyst_buf[4] = Opm::EclSphystGlobal::shiftSP;`
	`        }`
	
	`        const auto status = comm.broadcast(sphyst_buf, 5, 0);`
	`        if (status != MPI_SUCCESS) {`
	`            throw std::runtime_error(`
	`                "Unable to broadcast SPHYST parameters across MPI ranks"`
	`            );`
	`        }`
	
	`        // Unpack on all ranks (including rank 0)`
	`        const bool enabled = (sphyst_buf[0] != 0.0);`

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

	`Opm::EclSphystGlobal::enabled = enabled;`
	`Opm::EclSphystGlobal::alphaSP = sphyst_buf[1];`
	`Opm::EclSphystGlobal::betaSP  = sphyst_buf[2];`
	`Opm::EclSphystGlobal::gammaSP = sphyst_buf[3];`
	`Opm::EclSphystGlobal::shiftSP = sphyst_buf[4];`
	`}`
	
	`// In case of parse errors eclipseState/schedule might be null and`
	`// trigger segmentation faults in parallel during broadcast (e.g. when`
	`// serializing the non-existent TableManager)`

3.3 File: EclHysteresisConfig.hpp
Path: ~/opm/opm-common/opm/material/fluidmatrixinteractions/EclHysteresisConfig.hpp

Modifications:

Change at Line 94

Location: In enableWettingHysteresis() method
Change: Comparison operator changed from >= to ==

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)
`bool enableWettingHysteresis() const { return enableHysteresis() && krHysteresisModel_ >= 4; }`	`bool enableWettingHysteresis() const { return enableHysteresis()&& krHysteresisModel_ >= 4 && krHysteresisModel_ != 10; }`

3.4 File: EclHysteresisTwoPhaseLawParams.hpp
Path:
~/opm/opm-common/opm/material/fluidmatrixinteractions/EclHysteresisTwoPhaseLawParams.hpp

Modifications:

Change 1: Line 32 (New Include)

Location: After existing includes
Change: Added include for SPHYST global storage

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`#include <opm/input/eclipse/EclipseState/WagHysteresisConfig.hpp>`
 	
`#include <opm/material/common/EnsureFinalized.hpp>`
	`#include <opm/input/eclipse/EclipseState/WagHysteresisConfig.hpp>`
	
	`#include <opm/material/fluidmatrixinteractions/EclSphystGlobal.hpp>`
	
	`#include <opm/material/common/EnsureFinalized.hpp>`

Change 2: Lines 75-132 (9 New Methods)

Location: After maxPcog() method
Change: Added 9 accessor methods for SPHYST parameters (~60 lines)

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`return result;`
`}`
 	
`/*!`
` * \brief Calculate all dependent quantities...`
` */`
	`return result;`
	`}`
	
	`//==========`
	`// SPHYST SUPPORT (from global storage populated in readDeck.cpp)`
	`//==========`
	`bool sphystEnabled() const`
	`{ return Opm::EclSphystGlobal::enabled; }`
	
	`double alphaSP() const`
	`{ return Opm::EclSphystGlobal::alphaSP; }`
	
	`double betaSP() const`
	`{ return Opm::EclSphystGlobal::betaSP; }`
	
	`double gammaSP() const`
	`{ return Opm::EclSphystGlobal::gammaSP; }`
	
	`double shiftSP() const`
	`{ return Opm::EclSphystGlobal::shiftSP; }`
	
	`//==========`
	`// Spiteri-Blunt parameters: SPHYST-only`
	`//==========`
	`Scalar spiteriAlpha() const {`
	`  assert(sphystEnabled() && "SPHYST keyword is required...");`
	`  return Scalar(alphaSP());`
	`}`
	
	`Scalar spiteriBeta() const {`
	`  assert(sphystEnabled() && "SPHYST keyword is required...");`
	`  return Scalar(betaSP());`
	`}`
	
	`Scalar spiteriGamma() const {`
	`  assert(sphystEnabled() && "SPHYST keyword is required...");`
	`  return Scalar(gammaSP());`
	`}`
	
	`Scalar spiteriShift() const {`
	`  assert(sphystEnabled() && "SPHYST keyword is required...");`
	`  return Scalar(shiftSP());`
	`}`
	`//==========`

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

	`/*!`
	` * \brief Calculate all dependent quantities...`
	` */`

Changes 3-5: Adding Model 10 Checks

Locations: Lines 143, 197, and 464
Change: Added || config().krHysteresisModel() == 10 to three conditions
Pattern: Same modification applied in finalize() method (line 143), second condition in finalize() (line 197), and SwTrapped() method (line 464)

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`if (config().krHysteresisModel() == 2 ||`
`  config().krHysteresisModel() == 3 ||`
`  config().krHysteresisModel() == 4 ||`
`  config().pcHysteresisModel() == 0)`
`{`
	`if (config().krHysteresisModel() == 2 ||`
	`  config().krHysteresisModel() == 3 ||`
	`  config().krHysteresisModel() == 4 ||`
	`  config().krHysteresisModel() == 10 ||`
	`  config().pcHysteresisModel() == 0)`
	`{`

Change 6: Lines 794-809 (Spiteri Trapping)

Location: In trapped saturation calculation
Change: Complete Spiteri trapping model implementation

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`else {`
`Sncrt_ = Sncrd_;`
`}`
`}`
 	
`if (config().krHysteresisModel() == 4) {`
	`else {`
	`Sncrt_ = Sncrd_;`
	`}`
	`} else if (config().krHysteresisModel() == 10) {`
	`// Spiteri Trapping model using SPHYST parameters`
	`const Scalar snhy = 1.0 - krnSwMdc_;`
	`if (snhy > Sncrd_) {`
	
	`if (spiteriShift() == Scalar(0.0)) {`
	`// Option 1: The original Spiteri trapping function`
	`Sncrt_ = max(Scalar(0.0), min(snhy, snhy * (spiteriAlpha() - spiteriBeta() * snhy)));`
	`} else {`
	`// Option 2: Land Trapping function for Carlson`
	`Sncrt_ = snhy / (1 + spiteriShift() * snhy);`
	`}`
	`} else {`
	`Sncrt_ = Sncrd_;`
	`}`
	`}`
	
	`if (config().krHysteresisModel() == 4) {`

3.5 File: EclHysteresisTwoPhaseLaw.hpp
Path:
~/opm/opm-common/opm/material/fluidmatrixinteractions/EclHysteresisTwoPhaseLaw.hpp

Modifications:

Change 1: Lines 274-276

Location: In twoPhaseSatKrw() method, drainage curve condition
Change: Added support for krHysteresisModel == 10

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`if (params.config().krHysteresisModel() == 0 ||`
`  params.config().krHysteresisModel() == 2)`
`  // use drainage curve for wetting phase`
`  return EffectiveLaw::template twoPhaseSatKrw<...>(...);`
	`if (params.config().krHysteresisModel() == 0 ||`
	`  params.config().krHysteresisModel() == 2`
	`  ||`
	`params.config().krHysteresisModel() == 10)`
	`  // use drainage curve for wetting phase`
	`  return EffectiveLaw::template twoPhaseSatKrw<...>(...);`

Change 2: Lines 357-400 (Major Addition)

Location: In twoPhaseSatKrn() method
Change: Complete Spiteri-Blunt hysteresis model implementation (~40 lines)

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

`// Killough`
`assert(params.config().krHysteresisMode`
`l() == 2 ||`
 	
`params.config().krHysteresisModel() ==`
`3 ||`
 	
`params.config().krHysteresisModel() ==`
`4);`
 	
`Evaluation Snorm = params.Sncri() +`
`(1.0 - Sw - params.Sncrt()) *`
`(params.Snmaxd() - params.Sncri())`
`/`
`(params.Snhy() - params.Sncrt());`
 	
`return params.krnWght() *`
`EffectiveLaw::template`
`twoPhaseSatKrn<...>(`
`params.imbibitionParams(), 1.0`
`- Snorm);`
	`// Killough + Spiteri-Blunt`
	`assert(params.config().krHysteresisMode`
	`l() == 2 ||`
	
	`params.config().krHysteresisModel() ==`
	`3 ||`
	
	`params.config().krHysteresisModel() ==`
	`4 ||`
	
	`params.config().krHysteresisModel() ==`
	`10);`
	
	`Evaluation Snorm;`
	`if (params.config().krHysteresisModel()`
	`== 10) {`
	`// For primary drainage`
	`if (params.Snhy() < 0.0 || (1.0 -`
	`Sw) >= params.Snhy() - 1e-10) {`
	`return EffectiveLaw::template`
	`twoPhaseSatKrn<...>(...);`
	`}`
	`// Spiteri hysteresis model -`
	`parameters from SPHYST`
	`const Scalar epsilon = 1e-12;`
	`const Evaluation SnFree = max(1.0 -`
	`Sw - params.Sncrt(), epsilon);`
	`const Evaluation SnHist = 1.0 - Sw`
	`- params.Snhy();`
	`const Evaluation X = SnFree +`
	`params.spiteriGamma() * SnFree *`
	`SnHist;`
	
	`// Early exit for near-zero X or if`
	`Sn below trapped saturation`
	`if (abs(X) < epsilon || (1.0 - Sw)`
	`< params.Sncrt()) {`
	`return EffectiveLaw::template`
	`twoPhaseSatKrn<...>(..., 1.0);`
	`}`
	
	`const Evaluation a =`
	`params.spiteriAlpha() - 1.0;`
	`const Evaluation disc = max(a * a +`
	`4.0 * params.spiteriBeta() * X,`
	`epsilon);`
	`const Evaluation denom = sqrt(disc)`
	`- a;`
	
	`// Calculate Snf`

❌ ORIGINAL (Removed)	✅ MODIFIED (Added)

	`Evaluation Snf = (abs(denom) < epsilon) ? Evaluation(0.0) : (2.0 * X) / denom;`
	
	`if (Snf >= params.Snhy()) {`
	`return EffectiveLaw::template twoPhaseSatKrn<...>(..., Sw);`
	`}`
	
	`Snf = max(0.0, min(1.0 - Sw, Snf));`
	`return EffectiveLaw::template twoPhaseSatKrn<...>(..., 1.0 - Snf);`
	
	`} else {`
	`Snorm = params.Sncri() +`
	`(1.0 - Sw - params.Sncrt()) *`
	`(params.Snmaxd() - params.Sncri()) /`
	`(params.Snhy() - params.Sncrt());`
	
	`return params.krnWght() *`
	`EffectiveLaw::template twoPhaseSatKrn<...>(`
	`params.imbibitionParams(), 1.0 - Snorm);`
	`}`

==========
4. OPM FLOW INSTALLATION STEPS AFTER IMPLEMENTING SPITERI MODEL
==========

Run the following commands in Ubuntu:

4.1 Build opm-common

```
mkdir -p ~/opm/opm-common/build && cd ~/opm/opm-common/build

cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo

cmake --build . -j

sudo cmake --install .
```

4.2 Build opm-grid

```
mkdir -p ~/opm/opm-grid/build && cd ~/opm/opm-grid/build

cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo

cmake --build . -j

sudo cmake --install .
```

4.3 Build opm-simulators (Flow)

```
mkdir -p ~/opm/opm-simulators/build && cd ~/opm/opm-simulators/build

cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo

cmake --build . --target flow -j

sudo cmake --install .
```

==========
5. HOW SPHYST IS USED IN THE NEW SPITERI MODEL
==========

OPM input file (PROPS section)
Example:

```
PROPS


EHYSTR

1* 10 2* KR

/



SPHYST

0.8  0.45  2.0  0

/
```

Meaning:
Item 2 of EHYSTR → Selects the Spiteri hysteresis model (Use 10) for relative permeability in the gas phases; no hysteresis is applied to the water phase.
Item 1 of SPHYST → Alpha parameter in the Spiteri model.
Item 2 of SPHYST → Beta parameter in the Spiteri model.
Item 3 of SPHYST → Gamma parameter in the Spiteri model.
Item 4 of SPHYST → Use 0 for the original Spiteri model. Values greater than 0 activate the Carlson hysteresis model instead and assign the Land parameter C; in this case, Items 1, 2, and 3 are ignored.